CI Pipeline Design with Automated Testing

Day 4 · Module 4 · 2 Hours

SDET / DevOps Engineer

Intermediate

40% Theory / 60% Hands-on

TOOLS

Jenkins

GitHub Actions

Azure DevOps

LEARNING OUTCOMES

What You Will Achieve Today

Design CI pipelines with integrated testing

Define multi-stage pipelines in GitHub Actions, Jenkins, and Azure DevOps that execute automated tests as first-class gates.

Configure automated test execution stages

Add unit, integration, and regression test jobs to a CI pipeline — with correct triggers, parallel execution, and artefact publishing.

Handle test failures and flaky tests effectively

Interpret pipeline logs, trace failures to root cause, quarantine flaky tests, and enforce fail-fast vs fail-safe strategies.

CI Pipeline Stages, Triggers & Test Integration

Trigger
push / PR
schedule

→

Checkout
git clone
dependencies

→

Build
compile
package

→

Unit Tests
fast < 5 min
fail-fast

→

Integration Tests
API / service
parallel

→

Quality Gate
coverage
Sonar

→

Artefact
publish
report

Example Trigger Strategy (adapt to your team's cadence)

Push to any branch	Pull Request opened	Merge to main	Scheduled (nightly)
On every commit — runs unit tests only (fast loop)	Full test suite (unit + integration) — gates the merge button	Full regression + quality gate + artefact publish	Long-running E2E, cross-browser, performance baseline

Parallel vs Sequential Execution

Sequential	Parallel	Matrix	Fail-Fast
Stages run one after another. Safer dependency order. Slower total time. Use when stages share artefacts or state.	Independent jobs run concurrently on separate runners. Can substantially reduce wall-clock time when tests are independent and evenly distributed. Gain varies by setup overhead and runner availability.	Same job runs across multiple configs (OS, JDK, Node version) simultaneously. Ideal for cross-environment compatibility checks.	strategy: fail-fast: true cancels all matrix jobs when one fails. Saves compute. Risk: masks failures in other shards.

GitHub Actions — CI Pipeline with Test Execution

.github/workflows/ci.yml — unit test job

```yaml
name: CI — Build & Test

on:

  push:

  pull_request:

    branches: [main]


jobs:

  unit-test:

    runs-on: ubuntu-latest

    steps:

      - uses: actions/checkout@v6

      - uses: actions/setup-java@v5

        with:

          java-version: '21'

          distribution: temurin

      - name: Cache Maven dependencies

        uses: actions/cache@v4
```

Key Design Points

actions/checkout@v6 (current major)

Pin to a major version tag at minimum; use a full commit SHA for stronger supply-chain security in higher-risk environments. Avoid @main.

actions/cache@v4

Cache ~/.m2 or ~/.gradle keyed on pom.xml hash. Eliminates repeated dependency downloads — saves 1–3 min per run.

if: always()

Ensures test report artefacts upload even when the test step fails — critical for diagnosing CI failures.

actions/setup-java@v5 (current major)

Use distribution: temurin (Eclipse Adoptium). Specify java-version explicitly. Temurin is free, LTS-supported, widely used.

Secrets — never hardcode

Use ${{ secrets.MY_SECRET }} for credentials. Store in repo Settings → Secrets and variables → Actions.

Parallel & Matrix Test Execution in CI

75%

reduction in CI time — 40 min → 10 min with 4 shards

GitHub Actions matrix strategy: run the same job across N configurations simultaneously.

Set fail-fast: false so one shard failure doesn't cancel the rest — you want full visibility.

.github/workflows/ci.yml — parallel integration test sharding

```
integration-test:
  needs: unit-test          # wait for unit tests first
  runs-on: ubuntu-latest
  strategy:
    fail-fast: false        # all shards run even if one fails
    matrix:
      shard: [1, 2, 3, 4]  # 4 parallel runners
  steps:
    - uses: actions/checkout@v6
    - uses: actions/setup-java@v5
      with:
        java-version: '21'
        distribution: temurin
```

Cross-environment matrix (OS × Java version)

```
cross-env-test:
  strategy:
    matrix:
      os: [ubuntu-latest, windows-latest]
      java: ['17', '21']
  runs-on: ${{ matrix.os }}
  steps:
    - uses: actions/checkout@v6
    - uses: actions/setup-java@v5
      with:
        java-version: ${{ matrix.java }}
        distribution: temurin
    - run: mvn test
```

Jenkins — Declarative Pipeline with Test Stages

Jenkinsfile — Declarative Pipeline

```
pipeline {
  agent any
  tools { maven 'Maven-3.9' ; jdk 'JDK-21' }

  stages {
    stage('Checkout') {
      steps { checkout scm }
    }
    stage('Unit Tests') {
      steps {
        sh 'mvn test'
      }
      post {
        always {
          junit 'target/surefire-reports/**/*.xml'
        }
      }
```

Jenkins Key Concepts

Declarative vs Scripted

Declarative (pipeline {}) is the recommended approach — structured, readable, validates syntax at startup. Scripted (node {}) is Groovy-based, more flexible but harder to maintain.

agent directive

agent any: run on any available node. agent { docker 'maven:3.9-eclipse-temurin-21' } for containerised builds — ensures environment consistency.

junit step — publish results

junit 'target/surefire-reports/**/*.xml' publishes test results to Jenkins UI. Always place in post { always {} } so results publish even on test failure.

Jenkins vs GitHub Actions in 2025

GitHub Actions is commonly preferred for cloud-native and GitHub-first teams. Jenkins remains widely used for on-prem, compliance-heavy, or existing enterprise environments — the right choice depends on your context.

Azure DevOps — YAML Pipeline with Test Tasks

azure-pipelines.yml — multi-stage with test tasks

```yaml
trigger:
  branches:
    include: [main, 'feature/*']

pool:
  vmImage: ubuntu-latest

variables:
  MAVEN_OPTS: '-Xmx1024m'

stages:
- stage: CI
  displayName: Build & Test
  jobs:
  - job: UnitTest
    displayName: Unit Tests
    steps:
```

Azure DevOps Key Concepts

stages → jobs → steps

Three-level hierarchy. Stages group jobs. Jobs run on an agent in parallel by default. Steps are sequential tasks within a job.

Maven@4 task — publishJUnitResults

Set publishJUnitResults: true to publish results to the pipeline run summary and Tests tab in Azure Pipelines. Not the same as Azure Test Plans — results appear in the run UI, not a separate test management module.

condition: always()

Equivalent to if: always() in GitHub Actions. Ensures test result publishing and cleanup steps run even when earlier steps fail.

Azure DevOps vs GitHub Actions

Azure DevOps: richer enterprise features (Azure Boards, Test Plans, Artifacts), strong Microsoft ecosystem. GitHub Actions: simpler YAML, tighter GitHub integration, large Marketplace.

Handling Test Failures & Flaky Tests in CI

📊 Flaky tests are a major industry problem that consumes engineering time and obscures real regressions. Atlassian engineering: 15% of Jira backend CI failures were flaky tests; investigating them wasted 150,000+ dev-hours/year. The goal is not zero flakiness — keeping flake rate low enough that CI remains a reliable signal.

Reading Failed Build Logs

1. Check exit code

Non-zero exit = build failed. Unit test failures: Surefire returns exit 1. Integration: Failsafe checks during verify phase.

2. Read the summary

Surefire/Failsafe prints: Tests run: X, Failures: Y, Errors: Z, Skipped: N. Jump to FAILURE lines for the test name.

3. Find root assertion

Scroll past stack traces to the AssertionError or expected:<> but was:<> line. That is the actual failure, not the cascade above it.

4. Cross-reference commit

Use git log --oneline to trace which commit changed the failing class. git bisect automates binary search for the bad commit.

Flaky Test Management Strategy

Detect — do not ignore

Track pass/fail rate per test across runs. Any test that fails and passes on the same code is flaky. A low flake rate is essential for CI trust; many teams set internal targets such as < 2% per run.

Root causes to investigate

Common causes: async timing (test doesn't wait for async ops), shared mutable state (test order dependency), environment variance (CI ≠ dev machine), resource contention (parallel runners competing).

Quarantine — do not delete

Move flaky tests to a @Quarantine group or pytest.mark.flaky. They still run in a separate nightly job so they don't block PRs. Assign an owner and SLA for fix.

CI retry policy — use with discipline

GitHub Actions supports re-running failed jobs manually or via API. Automatic step-level retries require explicit scripting or third-party actions. Use sparingly — retries mask flakiness rather than fixing root cause.

DEMO

Demo: Create Pipeline → Run Tests → Analyse Failures

01 Creating a CI Pipeline with Test Execution Steps

- Create .github/workflows/ci.yml in the Day 2 framework repo
- Add checkout, Java setup, Maven cache, and mvn test steps
- Push to a feature branch — observe Actions tab in GitHub UI
- Walk through: job logs, step timings, artefact download

02 Adding Test Execution Steps & Viewing Pipeline Logs

- Add a second job integration-test with needs: unit-test
- Run mvn verify -Dgroups=integration — watch sequential dependency
- Open the test job log — locate pass/fail summary line
- Download surefire-reports artefact and open the XML in browser

03 Analysing Failed Builds — Trace to Root Cause

- Introduce a deliberate assertion failure in a test class
- Push commit — watch CI turn red within seconds
- Read the log: find exit code, FAILURE line, AssertionError
- Fix the assertion, push — verify CI returns green

HANDS-ON LAB

Lab: Configure, Trigger & Analyse

40% Theory | 60% Hands-on

Task 1

Working CI pipeline (green)

Configure CI Pipeline for Test Automation

1. Add .github/workflows/ci.yml to the Day 2 automation repo
2. Configure triggers: on push and pull_request to main
3. Add unit-test job: checkout → setup-java (temurin 21) → cache → mvn test
4. Push to a feature branch; verify Actions tab shows a green run

Task 2

Merged PR with green CI status

Trigger Pipeline on Code Commit & View Results

1. Open a PR from your feature branch to main
2. Observe: CI triggers automatically on PR open
3. View test artefact: download and open surefire-reports XML
4. Merge the PR — verify main branch CI also runs and passes

Task 3

Annotated log showing root cause

Analyse Failed Builds — Locate Root Cause

1. Introduce a deliberate failure: wrong assertion or missing import
2. Push commit — watch CI fail; note exact duration to first red signal
3. Navigate job log: find exit code, FAILURE summary, AssertionError line
4. Fix and push — confirm CI returns green; note recovery time

Day 4 Deliverables & Key Takeaways

Deliverables You Take Away

- CI Pipeline Configuration File (.github/workflows/ci.yml or Jenkinsfile or azure-pipelines.yml)
- Automated Test Execution Logs — green run screenshot + annotated failure log

Key Takeaways

1. A CI pipeline is test-driven: tests are the primary quality gate, not an afterthought
2. Trigger strategy matters — unit tests on push, full suite on PR, regression on merge
3. Matrix strategy and test sharding can significantly reduce CI time when tests are independent and evenly distributed across shards
4. fail-fast: false is often the right choice when full failure visibility matters — see all shard results before deciding what broke
5. Flaky tests erode CI trust. Detect, quarantine, assign an owner, and fix — do not just retry

Tomorrow — Day 5: API & Service-Level Continuous Testing · REST Assured / Postman / Newman in CI pipelines

Hands-on: Write API tests · integrate into pipeline · validate response data and schema

RECAP QUIZ

Module 4 – Quick Knowledge Check

Q1. What is the purpose of needs: in a GitHub Actions workflow?

- needs: defines a job dependency — the downstream job waits for the upstream job to complete successfully before starting. Use it to enforce stage ordering: unit tests must pass before integration tests run.

Q2. Why should upload-artifact and junit steps use if: always() / condition: always()?

- Without always(), report publishing and cleanup steps are skipped when a test step fails. always() ensures test results, logs, and artefacts are uploaded regardless of step outcome — critical for diagnosing CI failures.

Q3. What is the difference between fail-fast: true and fail-fast: false in a matrix strategy?

- fail-fast: true (default) cancels all remaining matrix jobs as soon as one fails — saves compute but hides failures in other shards. fail-fast: false lets all shards complete — gives full visibility of which configurations fail, at the cost of more runner minutes.

Q4. A test passes locally but fails intermittently in CI. Name two likely causes.

- Any two: async timing issues (test doesn't wait for async operations); shared/mutable test state (order-dependent tests); environment variance (different OS, timezone, or available memory in CI vs dev machine); resource contention between parallel runners.